

LAB REPORT 07

One-Dimensional Arrays in C++

Computer Programming · UET Nowshera · Electrical Engineering

1 Objective

The goal of this lab was to gain hands-on experience with one-dimensional arrays in C++. The session covered:

- Understanding what an array is and why it is a useful data structure
- Declaring and initialising integer and character arrays
- Reading and modifying individual elements through index-based access
- Copying array contents from one array to another using a loop
- Working with character arrays and accessing individual characters

2 Theoretical Background

What is an Array?

An **array** is a data structure that holds a fixed number of elements of the same data type under a single variable name. Instead of creating separate variables for each value, an array groups them together and lets each element be reached by a numeric *index*. In memory, all elements occupy consecutive locations with the first at the lowest address. Arrays make code cleaner and loops make it easy to process every element systematically.

Declaration and Initialisation

To declare an array, write the data type, a name, and the size in square brackets — e.g. `int age[4];`. Values can be provided at declaration time inside curly braces, or assigned element by element later using index notation. If fewer values than the declared size are provided, the remaining elements are zero-initialised automatically.

Accessing and Copying Elements

Each element is accessed using the array name followed by an index in square brackets. Indexing always starts at **0**, so for size *n* the valid indices are 0 to *n*-1. Since C++ has no built-in array-copy statement, elements must be copied individually inside a loop.

Character Arrays

A character array stores a sequence of characters to represent a word or short string. Each position holds exactly one character, accessible by the same index notation used for any other array type.

3 Software Used

IDE	DEV-C++ IDE
COMPILER	C++ Compiler (bundled with DEV-C++)

4 Experimental Tasks

Task 1 — Array Declaration and Initialisation

An integer array called numbers was declared with five elements and initialised with 10, 20, 30, 40, 50. A for loop steps through each index and prints the corresponding value.

● ● ● C++ Code

```
#include <iostream>
using namespace std;
int main() {
    int numbers[5] = {10, 20, 30, 40, 50};
    for (int i = 0; i < 5; i++)
        cout << "numbers[" << i << "] = " << numbers[i] << endl;
    return 0;
}
```

Task 2 — Accessing Character Array Elements

A character array named message stores the word "POWER". A for loop accesses each character by index and prints it on its own line.

● ● ● C++ Code

```
#include <iostream>
using namespace std;
int main() {
    char message[] = "POWER";
    for (int i = 0; i < 5; i++) { cout << message[i] << endl; }
    return 0;
}
```

Task 3 — Copying Arrays

Two integer arrays, source and destination, are declared with five elements each. A for loop transfers each element into the matching position of destination, confirmed by printing both arrays.

● ● ● C++ Code

```
#include <iostream>
using namespace std;
int main() {
    int source[5] = {5,10,15,20,25}, destination[5];
    for (int i = 0; i < 5; i++) destination[i] = source[i];
    cout << "Source:      ";
    for (int i = 0; i < 5; i++) cout << source[i] << " ";
    cout << "\nDestination: ";
    for (int i = 0; i < 5; i++) cout << destination[i] << " ";
    cout << endl; return 0;
}
```

Task 4 — Array Sum

The user enters five integers stored in an array. A second loop adds all five values and prints the total sum.

● ● ● C++ Code

```
#include <iostream>
using namespace std;
int main() {
    int nums[5], sum = 0;
    for (int i = 0; i < 5; i++) { cout << "Enter number " << i+1 << ": "; cin >> nums[i]
; }
    for (int i = 0; i < 5; i++) sum += nums[i];
    cout << "Sum = " << sum << endl; return 0;
}
```

5 Conclusion

This lab covered the essentials of one-dimensional arrays in C++: declaration, initialisation, index-based access, copying, and character arrays. The tasks made clear how tightly arrays and loops are connected — nearly every array operation requires a loop to traverse elements. These fundamentals are directly applicable when moving on to more complex data structures and algorithms.